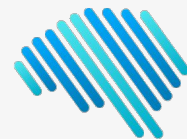




Pontificia Universidad Católica de Chile
Departamento de Ciencias de la Computación



IA Lab

Diplomado en Inteligencia Artificial

Herramientas de Deep Learning

Clase 8: Funciones de Pérdida, Regularización y Tareas Auxiliares

Carlos Aspillaga
Gabriel Sepúlveda

Contenidos

1 Funciones de Pérdida

- Contexto y Definición
- MSE
- Cross-Entropy

2 Regularización

- Motivación
- Regularización L2
- Regularización L1

3 Tareas Auxiliares

- Motivación
- Ejemplos
- Modo de Uso

4 Trabajo Práctico

- Funciones de Pérdida
- Regularización
- Tareas Auxiliares

Funciones de Pérdida Contexto

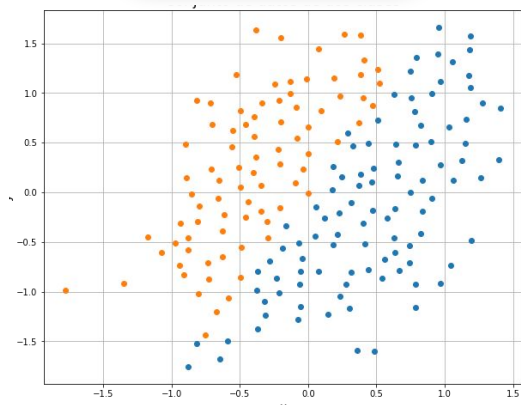
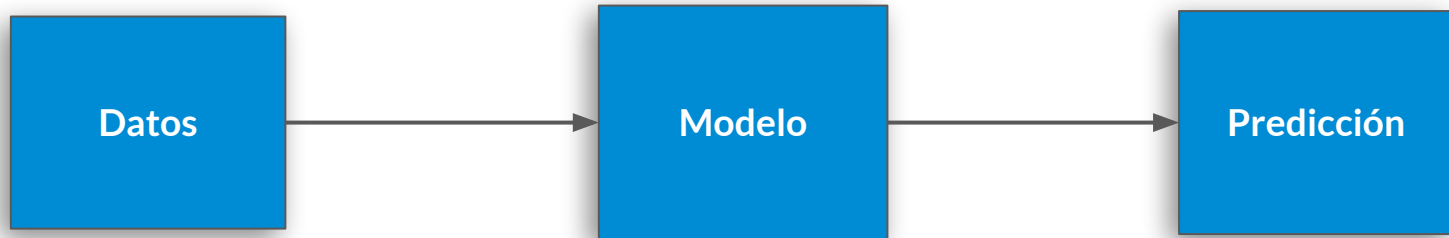
El problema de Machine Learning



x1	x2
0.75	0.1
1.2	0.57
0.87	0.3
2.5	1.1
...	...
0.2	0.4

Funciones de Pérdida Contexto

El problema de Machine Learning

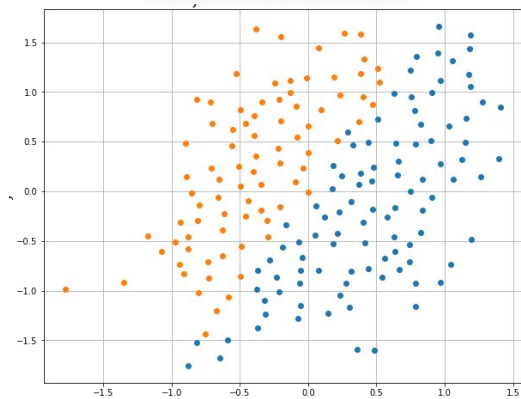
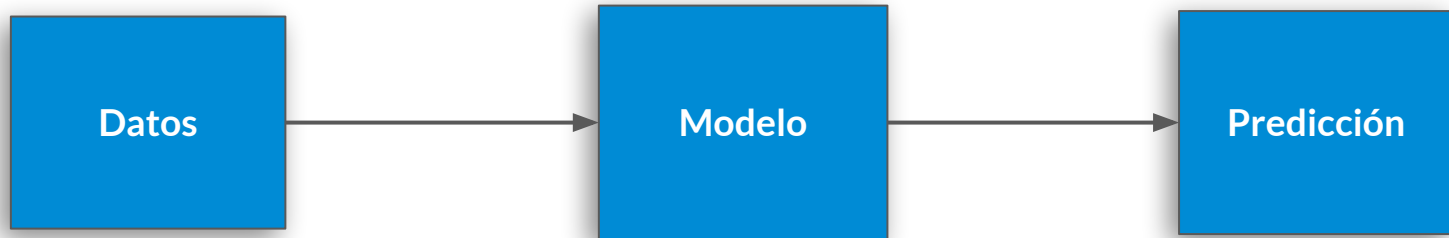


$A \cdot x_1 + B \cdot x_2 + C$
Representación /
Espacio de Hipótesis

> 0
 ≤ 0

Funciones de Pérdida Contexto

El problema de Machine Learning



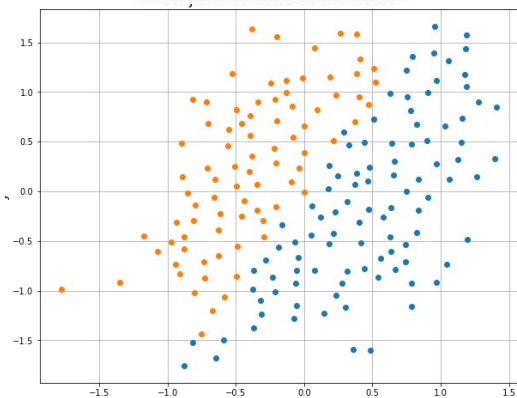
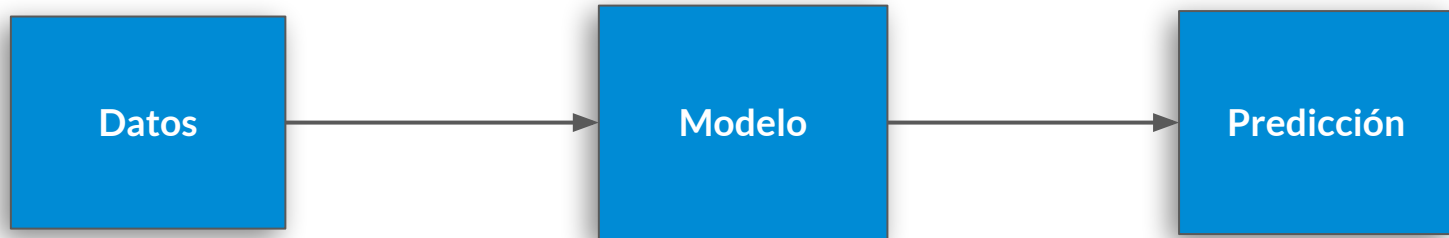
$A \cdot x_1 + B \cdot x_2 + C$
Representación /
Espacio de Hipótesis

> 0
 ≤ 0

¿Cómo le fue al modelo?
Función de Pérdida

Funciones de Pérdida Contexto

El problema de Machine Learning



$A \cdot x_1 + B \cdot x_2 + C$
Representación /
Espacio de Hipótesis

Corregir
Mecanismo de
Optimización

> 0
 ≤ 0

¿Cómo le fue al modelo?
Función de Pérdida

Función de Pérdida ¿Qué es?

Pensémosla como una métrica del “Error de Predicción”

*El **mejor modelo** es aquel que
minimiza la Función de Pérdida en promedio
para **todos los datos posibles**.*

Como difícilmente tenemos acceso a todos los datos posibles, se estima buscándolo en el set de entrenamiento.

Función de Pérdida ¿Qué es?

Caso General:

$$f^* \approx f_{Tr}^* = \arg \min_{f \in \mathcal{H}} \frac{1}{N} \sum_{x_i \in Tr} \mathcal{L}(f(x_i))$$

Caso Supervisado:

$$f^* \approx f_{Tr}^* = \arg \min_{f \in \mathcal{H}} \frac{1}{N} \sum_{x_i, y_i \in Tr} \mathcal{L}(f(x_i), y_i)$$



$$f(x_i, W)$$

Función de Pérdida MSE

Definición y ejemplo

$$\frac{1}{N} \sum_{x_i, y_i \in T_r} (\hat{y}_i - y_i)^2$$

Supongamos que tenemos sólo 3 ejemplos de entrenamiento.

Tenemos 2 modelos, que “en promedio” se equivocan por 5.0 puntos. ¿Cuál tiene mejor MSE?

	Real y_i	Modelo $\hat{y}_i$	Diferencia $\hat{y}_i - y_i$
Dato 1	5.0	0.0	5.0
Dato 2	10.0	10.0	0.0
Dato 3	20.0	10.0	10.0

Promedio: 5.0

	Real y_i	Modelo $\hat{y}_i$	Diferencia $\hat{y}_i - y_i$
Dato 1	5.0	10.0	5.0
Dato 2	10.0	5.0	5.0
Dato 3	20.0	15.0	5.0

Promedio: 5.0

Función de Pérdida MSE

Definición y ejemplo

$$\frac{1}{N} \sum_{x_i, y_i \in T_r} (\hat{y}_i - y_i)^2$$

Supongamos que tenemos sólo 3 ejemplos de entrenamiento.

Tenemos 2 modelos, que “en promedio” se equivocan por 5.0 puntos. ¿Cuál tiene mejor MSE?

	Real y_i	Modelo $\hat{y}_i$	Diferencia $\hat{y}_i - y_i$	$(\hat{y}_i - y_i)^2$
Dato 1	5.0	0.0	5.0	25
Dato 2	10.0	10.0	0.0	0
Dato 3	20.0	10.0	10.0	100
Promedio: 5.0				41.7

	Real y_i	Modelo $\hat{y}_i$	Diferencia $\hat{y}_i - y_i$	$(\hat{y}_i - y_i)^2$
Dato 1	5.0	10.0	5.0	25
Dato 2	10.0	5.0	5.0	25
Dato 3	20.0	15.0	5.0	25
Promedio: 5.0				25

Función de Pérdida MSE

Definición y ejemplo

$$\frac{1}{N} \sum_{x_i, y_i \in T_r} (\hat{y}_i - y_i)^2$$

- **MSE funciona bien** para problemas donde se desea aproximar una **variable continua** (ej: problema de regresión), o problemas donde nos **importa qué tan errado está el modelo** comparado con la respuesta ideal.
- Penaliza más las diferencias grandes.

¿Esto es bueno para problemas de clasificación?

Función de Pérdida MSE

Modo de Uso

¿Cómo se usa MSE en pytorch?



pytorch MSE



Todos

Imágenes

Noticias

Videos

Maps

Más

Preferencias

Herramientas

Cerca de 67,900 resultados (0.28 segundos)

[pytorch.org](#) > stable > generated > t... [Traducir esta página](#)

MSELoss — PyTorch 1.6.0 documentation

MSELoss. class torch.nn. MSELoss (size_average=None, reduce=None, reduction: str = 'mean') [source]. Creates a criterion that measures the mean squared ...

[pytorch.org](#) > docs > stable [Traducir esta página](#)

torch.nn — PyTorch 1.6.0 documentation

Creates a criterion that measures the **mean squared error** (squared L2 norm) between each element in the input x and target y . `nn.CrossEntropyLoss`.

[Torch.nn.functional](#) · [Torch.optim](#) · [Torch.nn.init](#) · [RNN](#)

[discuss.pytorch.org](#) > how-is-the-m... [Traducir esta página](#)

How is the MSELoss() implemented? - autograd - PyTorch ...

29 ene. 2018 - I'm trying to understand how MSELoss() is implemented. Usually people will think MSELoss is $(input - target)^2 \cdot \sum() / batch_size$, but when I ...

[pytorch.org](#) > docs > stable > nn.fu... [Traducir esta página](#)

torch.nn.functional — PyTorch 1.6.0 documentation

Measures the element-wise **mean squared error**. See MSELoss for details.
`margin_ranking_loss`, `torch.nn.functional`.

```
>>> import torch.nn as nn
>>> loss = nn.MSELoss()
>>> output = loss(predictions, target)
>>> output.backward()
```



pytorch MSE



Todos

Imágenes

Noticias

Videos

Maps

Más

Preferencias

Herramientas

Cerca de 67,900 resultados (0.28 segundos)

pytorch.org › stable › generated › t... ▼ Traducir esta página

MSELoss — PyTorch 1.6.0 documentation

MSELoss. class torch.nn. MSELoss (size_average=None, reduce=None, reduction: str = 'mean')
[source]. Creates a criterion that measures the **mean squared** ...

pytorch.org › docs › stable ▼ Traducir esta página

torch.nn — PyTorch 1.6.0 documentation

Creates a criterion that measures the **mean squared error** (squared L2 norm) between each element in the input x and target y . `nn.CrossEntropyLoss`.
[Torch.nn.functional](#) · [Torch.optim](#) · [Torch.nn.init](#) · [RNN](#)

discuss.pytorch.org › how-is-the-m... ▼ Traducir esta página

How is the MSELoss() implemented? - autograd - PyTorch ...

29 ene. 2018 - I'm trying to understand how MSELoss() is implemented. Usually people will think MSELoss is $(input - target)^2 \cdot \sum() / batch_size$, but when I ...

pytorch.org › docs › stable › nn.fu... ▼ Traducir esta página

torch.nn.functional — PyTorch 1.6.0 documentation

Measures the element-wise **mean squared error**. See MSELoss for details.
[margin_ranking_loss](#), [torch.nn.functional](#).

```
>>> import torch.nn.functional as F
>>> output = F.mse_loss(predictions, target)
>>> output.backward()
```

Función de Pérdida Cross-Entropy

Definición

Proviene del estudio de *“Teoría de la Información”*

*“Cross-entropy **is the overhead** of transmitting a signal defined by a **pdf $p(x)$** using an encoding that considers a **pdf $q(x)$** ”*

En Machine Learning usamos esta *“métrica de ineficiencia”* como score de distancia entre dos distribuciones de probabilidad.

Función de Pérdida Cross-Entropy

Definición

$\begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$

Distribución de probabilidad “*real*” $p(x_i) := y_i$



Distribución de probabilidad “*estimada*” $q(x_i) := \hat{y}_i$



$\begin{pmatrix} 0.01 \\ 0.02 \\ 0.89 \\ 0.0 \\ 0.03 \\ 0.01 \\ 0.01 \\ 0.02 \\ 0.0 \\ 0.01 \end{pmatrix}$

$$\text{Cross-Entropy} = - \sum_{x_i, y_i \in T} y_i * \log \hat{y}_i$$

Función de Pérdida Cross-Entropy

Definición



$$\text{Cross-Entropy} = - \sum_{x_i, y_i \in T} y_i * \log \hat{y}_i$$

0.01
0.02
0.89
0.0
0.03
0.01
0.01
0.02
0.0
0.01

Función de Pérdida Cross-Entropy

Definición



Los frameworks como pytorch manejan esto internamente

$$\text{Cross-Entropy} = - \sum_{x_i, y_i \in T} y_i * \log \hat{y}_i$$



$\begin{pmatrix} 0.01 \\ 0.02 \\ 0.89 \\ 0.0 \\ 0.03 \\ 0.01 \\ 0.01 \\ 0.02 \\ 0.0 \\ 0.01 \end{pmatrix}$

+ 1e-10

Función de Pérdida Cross-Entropy

Definición

$\begin{pmatrix} -58.0 \\ 18.3 \\ 0.008 \\ 0.935 \\ -0.156 \\ -88.72 \\ 0.01 \\ 10.24 \\ 3.333 \\ 2.5 \end{pmatrix}$



EXPECTATION...	
REALITY...	



$\begin{pmatrix} 0.000000 \\ 0.999684 \\ 0.000000 \\ 0.000000 \\ 0.000000 \\ 0.000000 \\ 0.000000 \\ 0.000000 \\ 0.000316 \\ 0.000000 \\ 0.000000 \end{pmatrix}$

Función de Pérdida Cross-Entropy + Softmax

Definición

$\begin{pmatrix} -58.0 \\ 18.3 \\ 0.008 \\ 0.935 \\ -0.156 \\ -88.72 \\ 0.01 \\ 10.24 \\ 3.333 \\ 2.5 \end{pmatrix}$



softmax

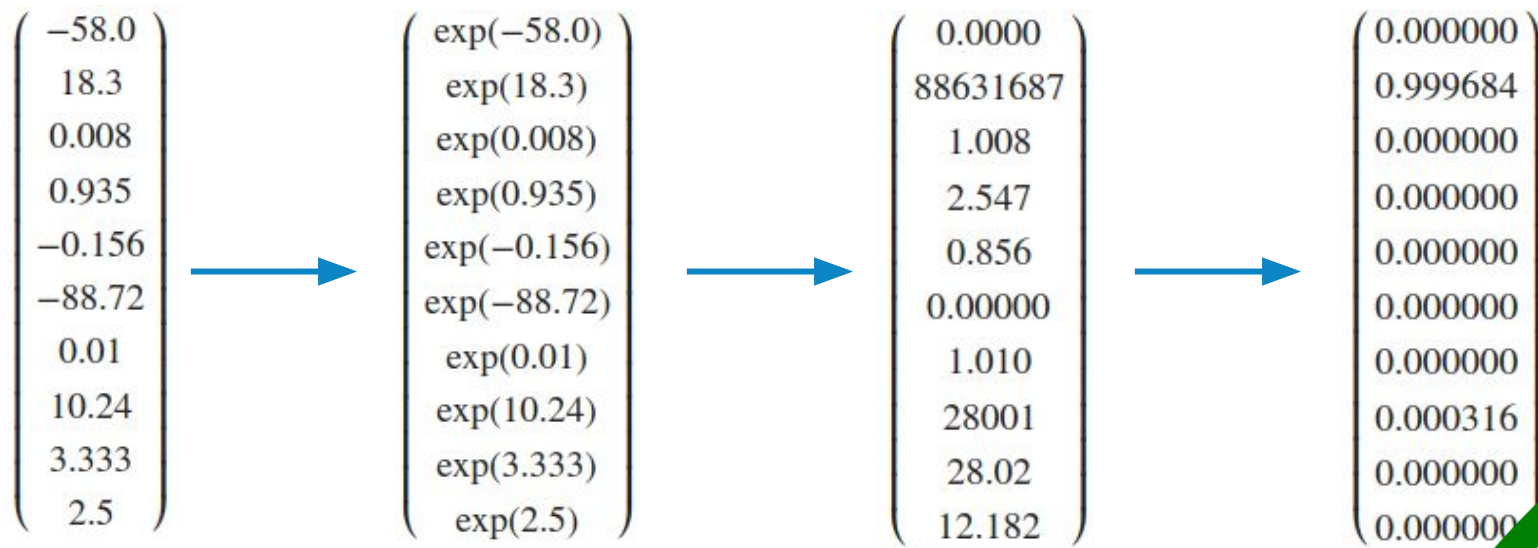
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



$\begin{pmatrix} 0.000000 \\ 0.999684 \\ 0.000000 \\ 0.000000 \\ 0.000000 \\ 0.000000 \\ 0.000000 \\ 0.000316 \\ 0.000000 \\ 0.000000 \end{pmatrix}$

Función de Pérdida Cross-Entropy + Softmax

Definición



Función de Pérdida Cross-Entropy + Softmax

La combinación perfecta



Softmax no sólo normaliza la salida:

- ✓ Es diferenciable
- ✓ Acentúa el máximo y castiga todo lo demás (de ahí el nombre soft-max)
- ✓ Se combina muy bien con Cross-Entropy

$$\log(\text{softmax}(z_i)) = z_i - \log \sum_j e^{z_j}$$

al maximizar, se
intentará maximizar este
término directamente

... y minimizar la suma
de todos los demás

- ✓ Combinada con Cross-Entropy la optimización es mucho más **rápida de calcular** que MSE.

Función de Pérdida Cross-Entropy

Modo de Uso

¿Cómo se usa Cross-Entropy en pytorch?

Cerca de 149,000 resultados (0.27 segundos)

pytorch.org > master > generated ▼ Traducir esta página

CrossEntropyLoss — PyTorch master documentation

CrossEntropyLoss. class torch.nn. CrossEntropyLoss (weight: Optional[torch.Tensor] = None, size_average=None, ignore_index: int = -100, reduce=None, ...

pytorch.org > docs > stable ▼ Traducir esta página

torch.nn — PyTorch 1.6.0 documentation

Creates a criterion that measures the Binary **Cross Entropy** between the target and the output: nn.BCEWithLogitsLoss. This loss combines a Sigmoid layer and ...

Torch.nn.functional · Torch.optim · Torch.nn.init · RNN

discuss.pytorch.org > usage-of-cros... ▼ Traducir esta página

Usage of cross entropy loss - PyTorch Forums

13 mar. 2018 - Is **cross entropy** loss good for multi-label classification or for binary-class classification? Please also tell how to use it? criterion = nn.

discuss.pytorch.org > categorical-cr... ▼ Traducir esta página

Categorical cross entropy loss function equivalent in PyTorch ...

Hi, I found Categorical **cross-entropy** loss in Theano and Keras. Is nn.CrossEntropyLoss() equivalent of this loss function? I saw this topic but three is not a ...

1 respuesta

pytorch.org > torch.nn.BCELoss.html ▼ Traducir esta página

BCELoss — PyTorch master documentation

Creates a criterion that measures the Binary **Cross Entropy** between the target and the output: The unreduced (i.e. with reduction set to 'none') loss can be ...

pytorch.org > docs > stable > nn.fu... ▼ Traducir esta página

torch.nn.functional — PyTorch 1.6.0 documentation

Function that measures Binary **Cross Entropy** between target and output logits. See BCEWithLogitsLoss for details. Parameters. input – Tensor of arbitrary shape.

```
>>> import torch.nn as nn
>>> loss = nn.CrossEntropyLoss()
>>> output = loss(predictions, targets)
>>> output.backward()
```

```
>>> import torch.nn as nn
>>> loss = nn.CrossEntropyLoss(weight)
>>> output = loss(predictions, targets)
>>> output.backward()
```

OJO: Esta implementación tiene incorporada la Softmax.

Parámetro “weight” es útil para datasets desbalanceados.

Tiene su análogo en “functional”:

F.cross_entropy

Cerca de 149,000 resultados (0.27 segundos)

pytorch.org > master > generated ▼ Traducir esta página

CrossEntropyLoss — PyTorch master documentation

CrossEntropyLoss. class torch.nn. CrossEntropyLoss (weight: Optional[torch.Tensor] = None, size_average=None, ignore_index: int = -100, reduce=None, ...

pytorch.org > docs > stable ▼ Traducir esta página

torch.nn — PyTorch 1.6.0 documentation

Creates a criterion that measures the Binary **Cross Entropy** between the target and the output: nn.BCEWithLogitsLoss. This loss combines a Sigmoid layer and ...

Torch.nn.functional · Torch.optim · Torch.nn.init · RNN

discuss.pytorch.org > usage-of-cros... ▼ Traducir esta página

Usage of cross entropy loss - PyTorch Forums

13 mar. 2018 - Is **cross entropy** loss good for multi-label classification or for binary-class classification? Please also tell how to use it? criterion = nn.

discuss.pytorch.org > categorical-cr... ▼ Traducir esta página

Categorical cross entropy loss function equivalent in PyTorch ...

Hi, I found Categorical **cross-entropy** loss in Theano and Keras. Is nn.CrossEntropyLoss() equivalent of this loss function? I saw this topic but three is not a ...

1 respuesta

pytorch.org > torch.nn.BCELoss.html ▼ Traducir esta página

BCELoss — PyTorch master documentation

Creates a criterion that measures the Binary **Cross Entropy** between the target and the output: The unreduced (i.e. with reduction set to 'none') loss can be ...

pytorch.org > docs > stable > nn.fu... ▼ Traducir esta página

torch.nn.functional — PyTorch 1.6.0 documentation

Function that measures Binary **Cross Entropy** between target and output logits. See BCEWithLogitsLoss for details. Parameters. input – Tensor of arbitrary shape.

```
>>> import torch.nn as nn
>>> s = nn.Sigmoid()
>>> loss = nn.BCELoss()
>>> output = loss(s(predictions), targets)
>>> output.backward()
```

```
>>> import torch.nn as nn
>>> s = nn.Sigmoid()
>>> loss = nn.BCELoss(weight)
>>> output = loss(s(predictions), targets)
>>> output.backward()
```

Parámetro “weight” es útil para datasets desbalanceados.

Tiene su análogo en “functional”:

F.binary_cross_entropy y **F.sigmoid**

Cerca de 149,000 resultados (0.27 segundos)

pytorch.org > master > generated ▼ Traducir esta página

CrossEntropyLoss — PyTorch master documentation

CrossEntropyLoss. class torch.nn. CrossEntropyLoss (weight: Optional[torch.Tensor] = None, size_average=None, ignore_index: int = -100, reduce=None, ...

pytorch.org > docs > stable ▼ Traducir esta página

torch.nn — PyTorch 1.6.0 documentation

Creates a criterion that measures the Binary **Cross Entropy** between the target and the output: nn.BCEWithLogitsLoss. This loss combines a Sigmoid layer and ...

Torch.nn.functional · Torch.optim · Torch.nn.init · RNN

discuss.pytorch.org > usage-of-cros... ▼ Traducir esta página

Use of cross entropy loss - PyTorch Forums

13 mar. 2018 - Is **cross entropy** loss good for multi-label classification or for binary-class classification? Please also tell how to use it? criterion = nn.

discuss.pytorch.org > categorical-cr... ▼ Traducir esta página

Categorical cross entropy loss function equivalent in PyTorch ...

Hi, I found Categorical **cross-entropy** loss in Theano and Keras. Is nn.CrossEntropyLoss() equivalent of this loss function? I saw this topic but three is not a ...

1 respuesta

pytorch.org > torch.nn.BCELoss.html ▼ Traducir esta página

BCELoss — PyTorch master documentation

Creates a criterion that measures the Binary **Cross Entropy** between the target and the output: The unreduced (i.e. with reduction set to 'none') loss can be ...

pytorch.org > docs > stable > nn.fu... ▼ Traducir esta página

torch.nn.functional — PyTorch 1.6.0 documentation

Function that measures Binary **Cross Entropy** between target and output logits. See BCEWithLogitsLoss for details. Parameters. input – Tensor of arbitrary shape.

```
>>> import torch.nn as nn
>>> loss = nn.BCEWithLogitsLoss()
>>> output = loss(predictions, targets)
>>> output.backward()
```

```
>>> import torch.nn as nn
>>> loss = nn.BCEWithLogitsLoss(weight)
>>> output = loss(predictions, targets)
>>> output.backward()
```

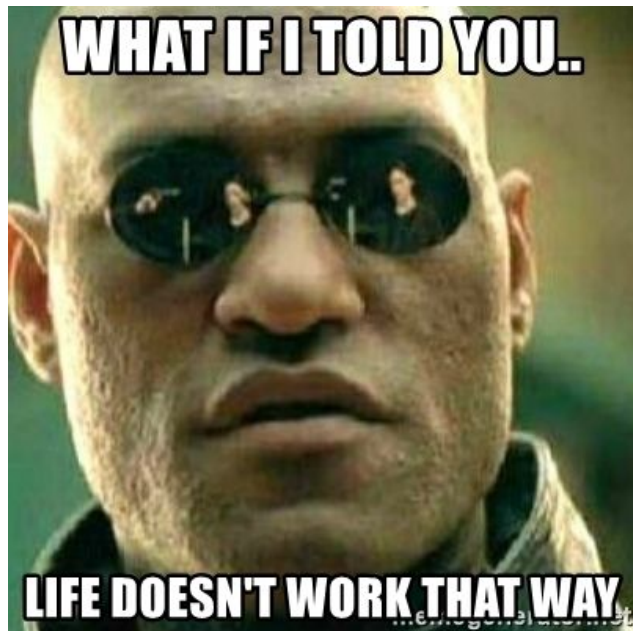
OJO: Esta implementación tiene incorporada la Sigmoide.

Parámetro “weight” es útil para datasets desbalanceados.

Tiene su análogo en “functional”:

F.binary_cross_entropy_with_logits

En la práctica no todo es tan sencillo
Herramientas para combatirlo



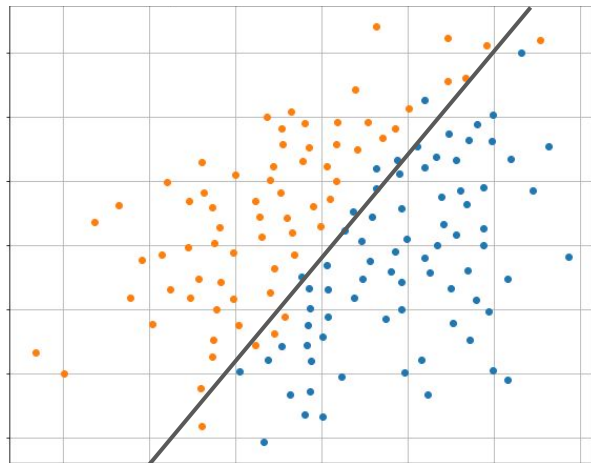
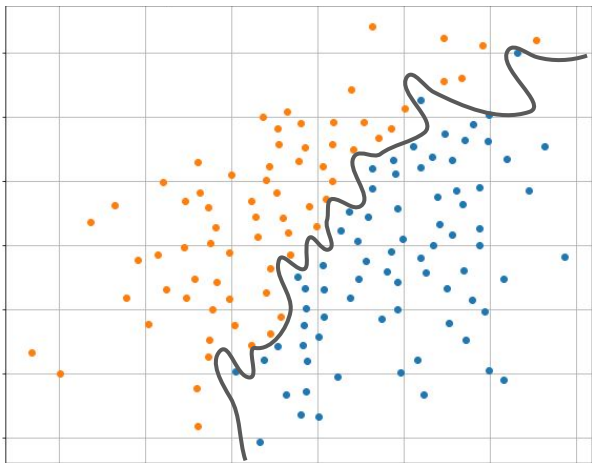
El modelo se veía bien en teoría, pero...

- ... no aprende?
- ... aprendió una correlación incorrecta?
- ... no generaliza / overfitting?
- ... no alcanza el rendimiento que quisiera?

Regularización

Motivación

- **Restringir el espacio de hipótesis** a un espacio que usualmente funciona mejor.
- La manera más básica de regularizar es simplemente **limitar la capacidad del modelo**.
- Regularizar nos ayuda a evitar tener que simplificar el modelo.



Regularización L2 / Weight-Decay

Definición

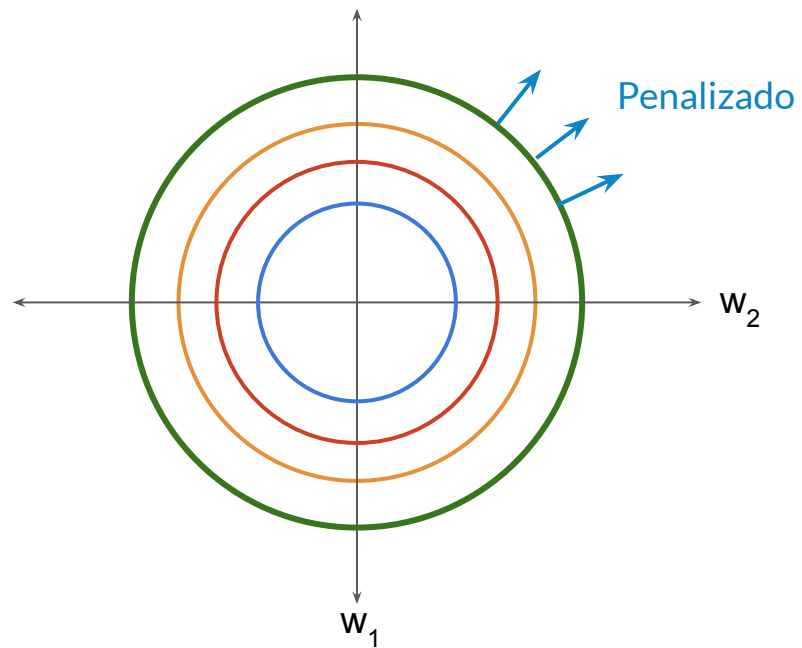
- **Idea: Castigar pesos con valores “grandes”.**
 - Pesos grandes **amplifican el ruido**.
 - Pesos pequeños generan superficies de decisión más **suaves**.
 - Priorizar pesos pequeños obliga al modelo a **distribuir su decisión** en varios pesos, en lugar de tener pocos pesos dominantes.

En la práctica es simplemente **sumar un componente a la función de pérdida** que castigue el valor de los pesos w , con un **parámetro λ para manejar su relevancia**.

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N w_i^2$$

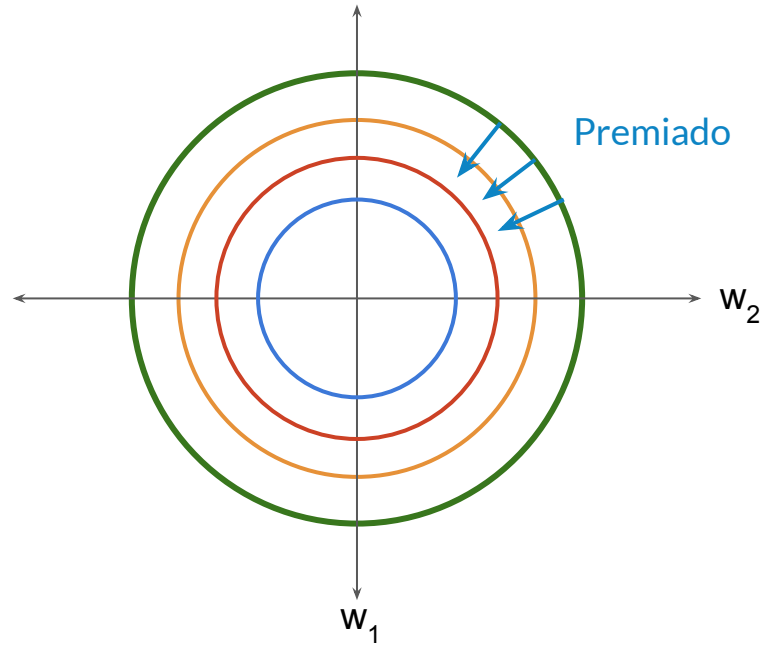
Regularización L2 / Weight-Decay

Representación gráfica



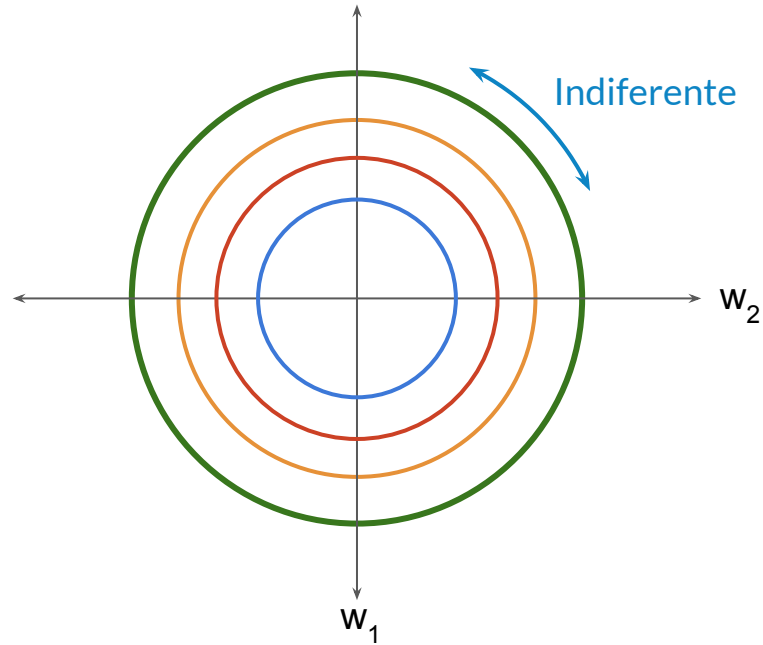
Regularización L2 / Weight-Decay

Representación gráfica



Regularización L2 / Weight-Decay

Representación gráfica



Regularización L1

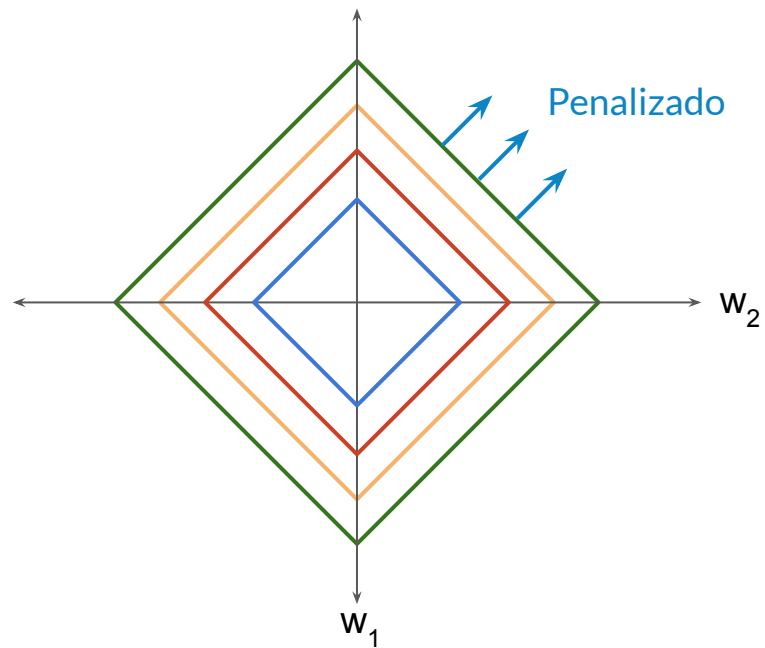
Definición

Similar a L2, pero **penalizando la suma absoluta**.

$$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^N |w_i|$$

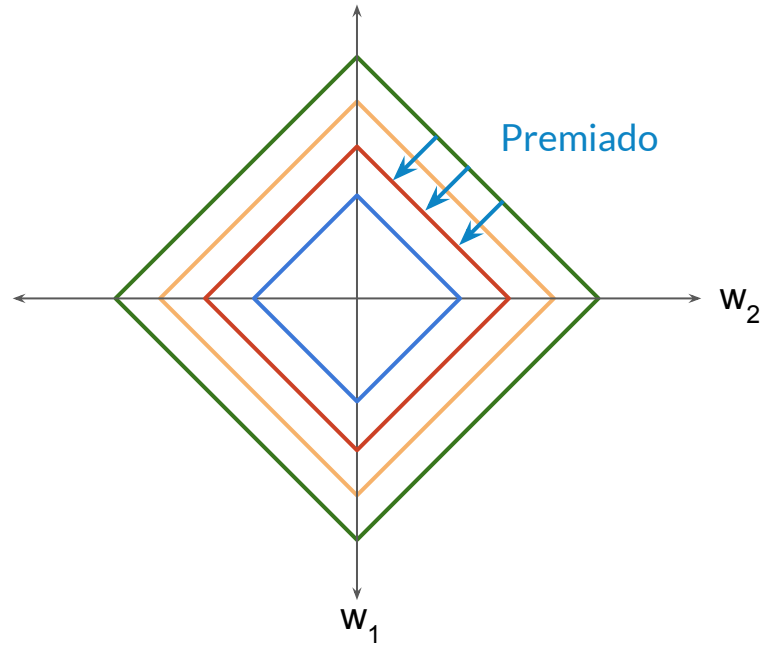
Regularización L1

Representación gráfica



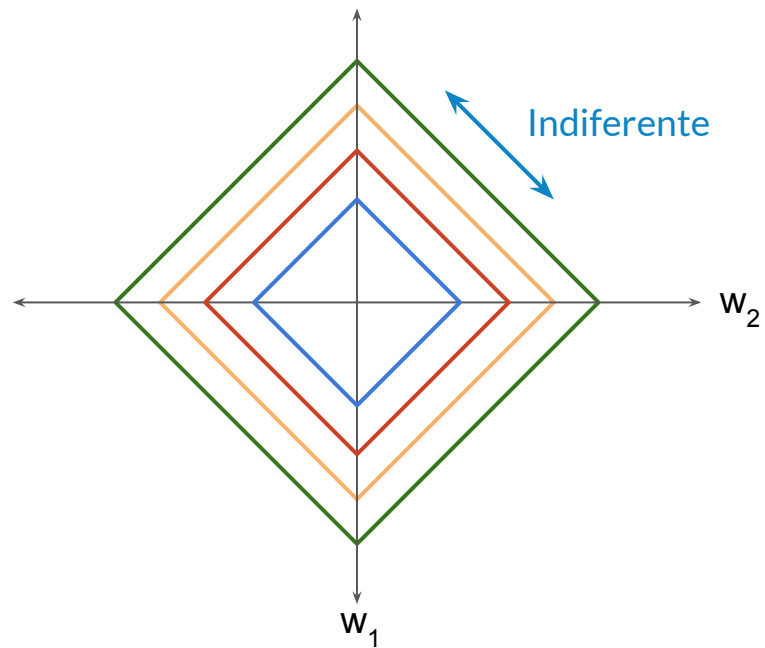
Regularización L1

Representación gráfica



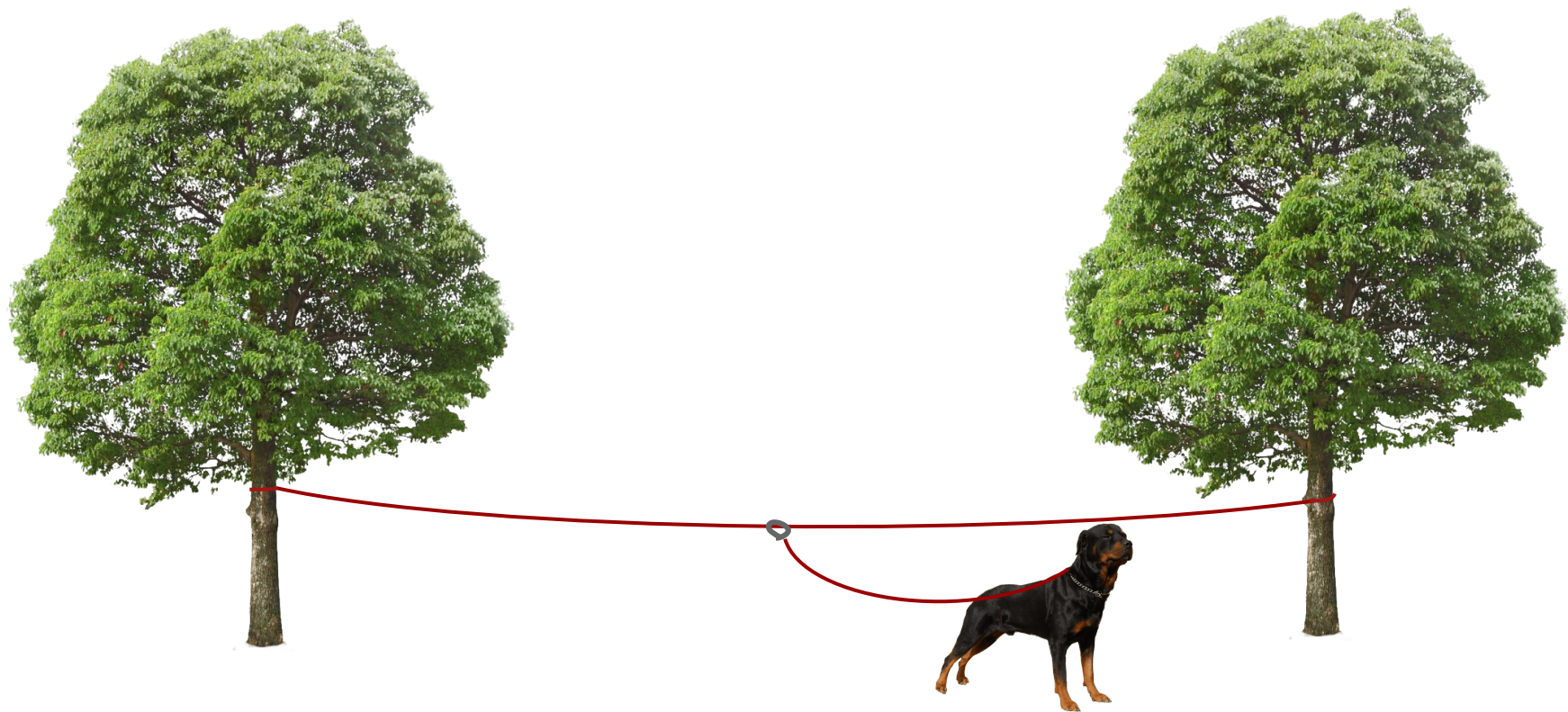
Regularización L1

Representación gráfica



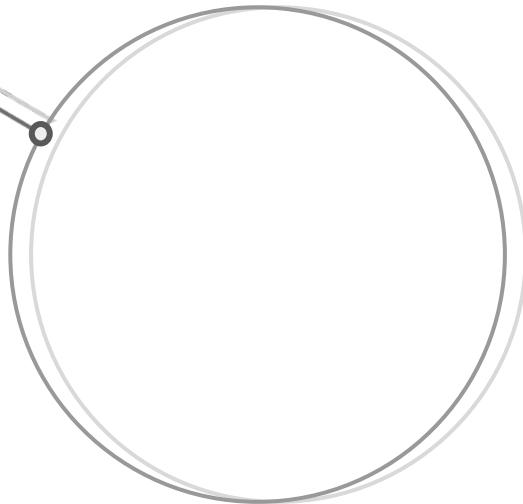
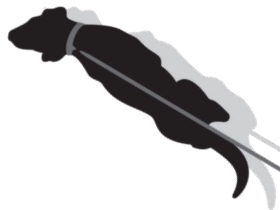
Regularización

Entendamos el efecto de usar L1 o L2



Regularización L2

Representación gráfica



Regularización L2

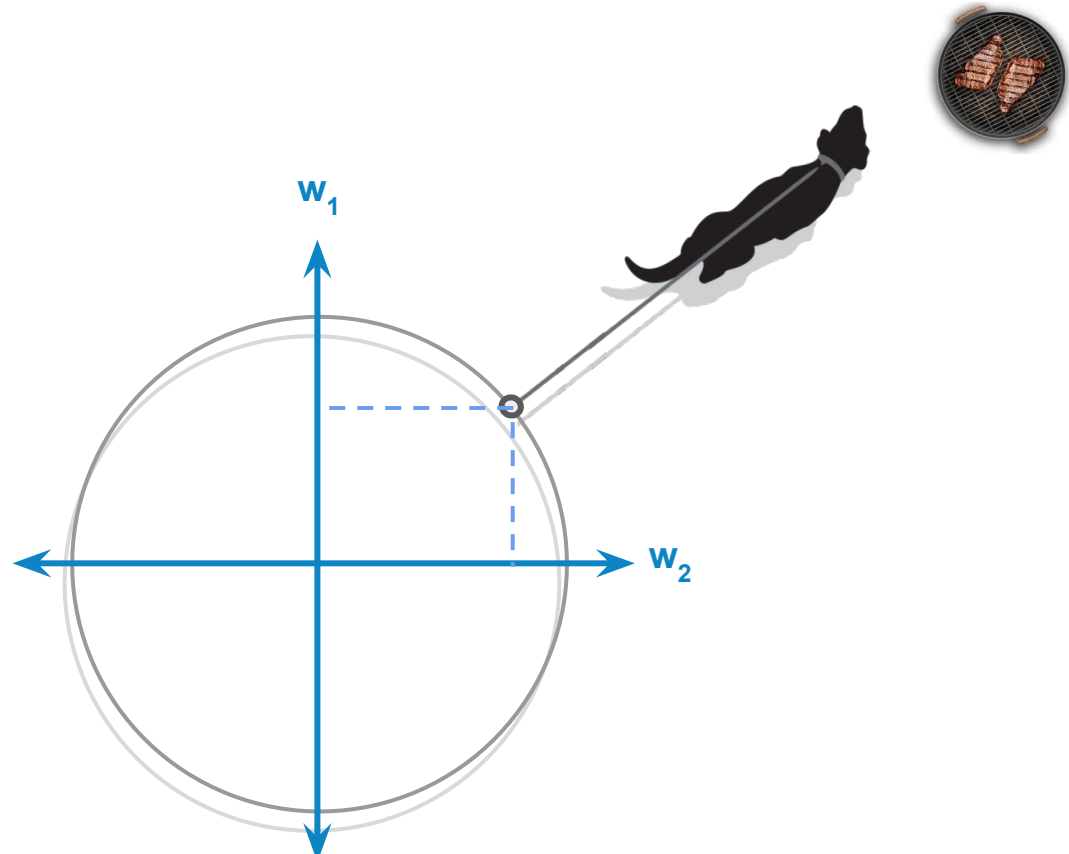
Representación gráfica



Regularización L2

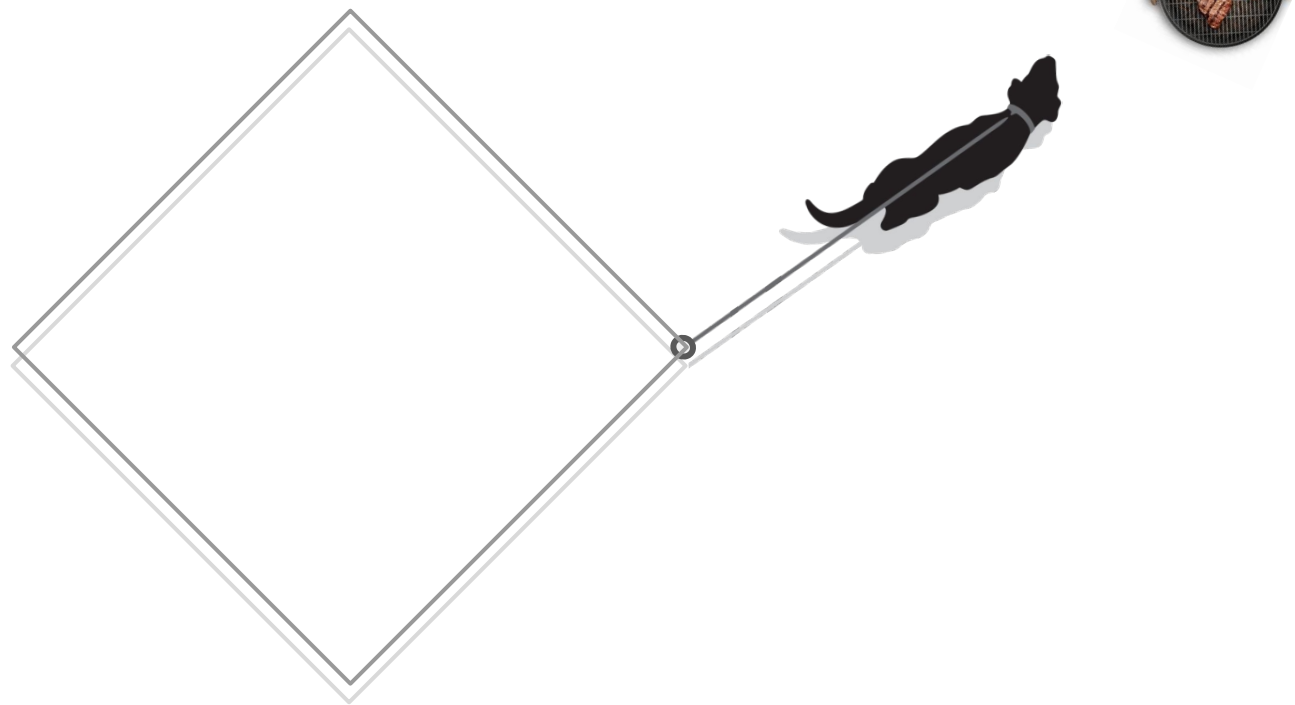
Representación gráfica

Todos los pesos suelen tomar algún valor



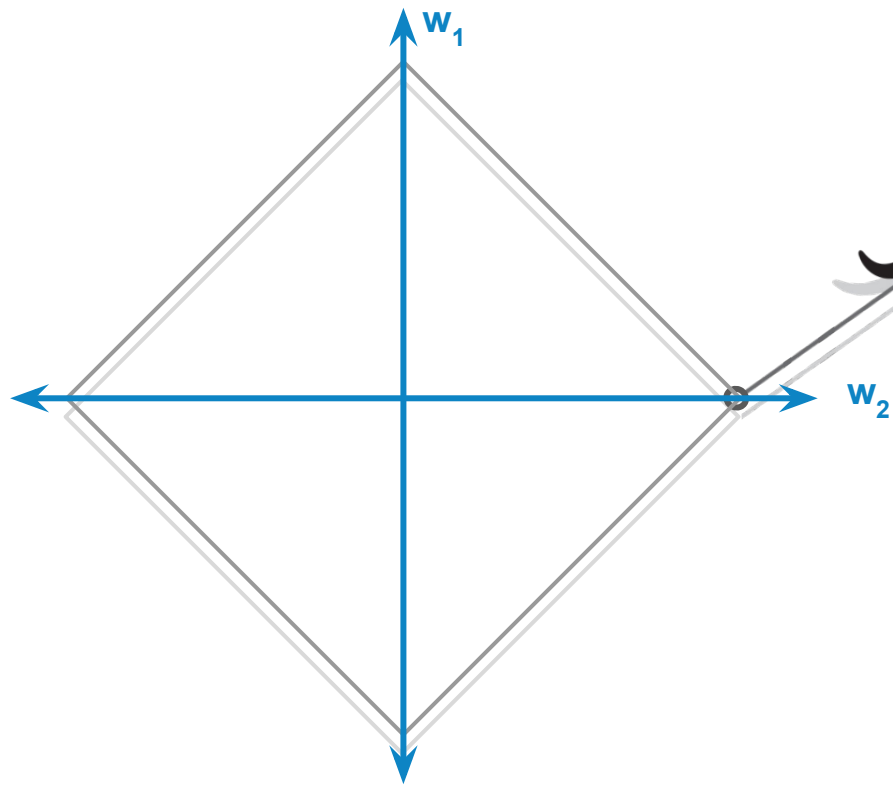
Regularización L1

Representación gráfica



Regularización L1

Representación gráfica



Tal como w_1 , varios pesos
suelen tomar valor cero
(feature selection)

Tareas Auxiliares

Motivación



Tareas Auxiliares

Una ayuda indirecta

A veces uno tiene la **intuición** de algunas **tareas complementarias** que podrían **potenciar al modelo** si fuera capaz de dominarlas, aunque no se relacionen directamente con la tarea principal.



Tareas Auxiliares

Una ayuda indirecta

A veces el modelo no aprende lo que queríamos



Tareas Auxiliares

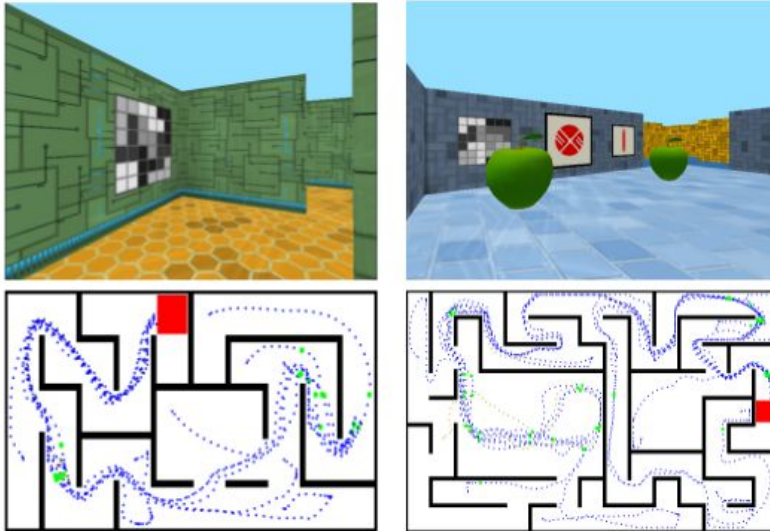
Una ayuda indirecta

A veces el modelo no aprende lo que queríamos



Tareas Auxiliares

Algunas tareas le permiten mejorar su representación del mundo



Tarea principal:

Recorrer un laberinto

Tareas complementarias:

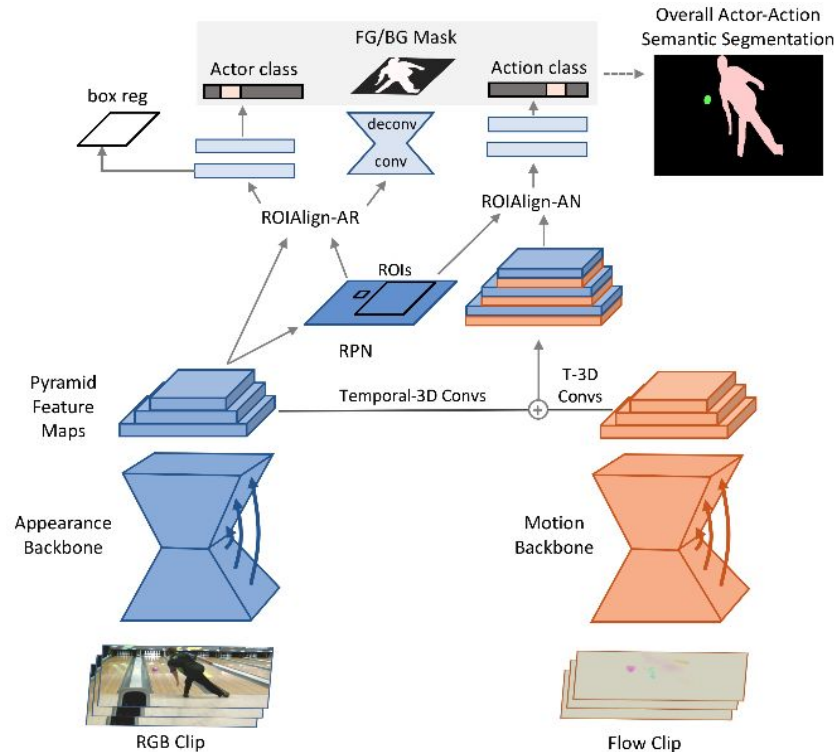
Estimar la distancia al muro

Detectar cierre de loops

P. Mirowski et al., Learning to Navigate in Complex Environments, ICLR 2017

Tareas Auxiliares

Algunas tareas le permiten mejorar su representación del mundo



Tarea principal:

Clasificación de acciones en video

Tareas complementarias:

Bounding-box del actor

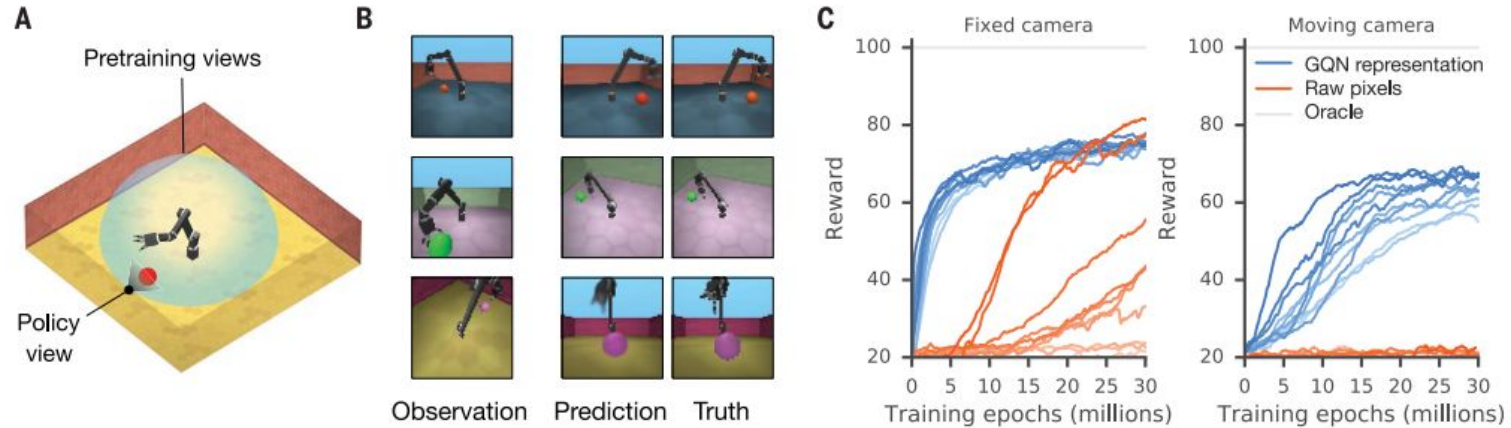
Clasificación del actor

Segmentación del actor

J. Ji et al., End-to-End Joint Semantic Segmentation of Actors and Actions in Video, ECCV 2018

Tareas Auxiliares

Algunas tareas permiten que el modelo aprenda cosas de las que antes era incapaz



Tarea principal:

Mover un brazo robótico

Tarea complementaria:

Generar una vista desde otra perspectiva

Eslami et al., Neural Scene Representation and Rendering, Science Magazine, 2018

Tareas Auxiliares

¿Cómo se incorporan en la práctica?

$$Loss = Error_0(y_0, \hat{y}_0) + \lambda_1 Error_1(y_1, \hat{y}_1) + \lambda_2 Error_2(y_2, \hat{y}_2) + \dots + \lambda_n Error_n(y_n, \hat{y}_n)$$

Feedback anónimo

Las 3 preguntas más importantes de la clase

<https://b.socrative.com/login/student/>



Student Login

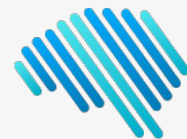
Room Name

ASPILLAGA4977

JOIN



Pontificia Universidad Católica de Chile
Departamento de Ciencias de la Computación



IA Lab

Diplomado en Inteligencia Artificial

Herramientas de Deep Learning

Funciones de Pérdida, Regularización y Tareas Auxiliares

Carlos Aspillaga
Gabriel Sepúlveda